Compression

trade CPU for bytes, when it pays

Watch the 4-min video

In one line: compression spends CPU to send and store fewer bytes; it wins on repetitive text when the network or disk is the bottleneck; use Brotli or gzip for HTTP (low levels for dynamic responses, max level precompressed for static files) and zstd for storage and internal traffic, and skip it for already-compressed or tiny data, CPU-bound servers, or secrets mixed with user input.

1 Two megabytes on a slow phone

Your API returns a 2 MB JSON product list. Mobile users on slow networks wait seconds for it. Turn on compression and it becomes about 200 KB. But compression costs CPU, and sometimes it isn't worth it.

2 CPU for bytes

The server spends CPU to shrink the data, the network carries fewer bytes, and the client spends CPU to decompress. It wins when the **network is the bottleneck** and the data is **repetitive**: JSON, HTML, logs, CSV. The **compression ratio** measures how much smaller it gets; text often shrinks to a fifth or a tenth.



text (JSON, HTML, CSV, logs) often shrinks to 1/5 – 1/10

Figure 1. Compression moves cost from the network to the CPUs at both ends.

3 Three algorithms

algorithm	ratio	speed	best for
gzip	good	medium	universal HTTP; every client
Brotli (br)	~15–20% smaller than gzip on text	slow at max level	HTTP text, precompressed static files
zstd	excellent	very fast	storage, logs, Kafka, internal traffic

Figure 2. gzip, Brotli and zstd compared.

Each has **levels**: higher levels squeeze more but cost much more CPU. For **dynamic** responses use a low or medium level (gzip 6 is a common default; Brotli 4–5), so compression doesn't add latency. For **static** assets, compress once at build time with the highest level (Brotli 11) and serve the precompressed file. In HTTP, the client sends `Accept-Encoding: br, gzip`, and the server replies with `Content-Encoding: br` (and `Vary: Accept-Encoding`, lesson 31).

4 Compression in storage

Columnar formats like **Parquet** store each column together, and similar values compress extremely well, often ten times or more. **Kafka** compresses whole batches of messages (lesson 39). Databases compress pages and backups. Less data on disk means less I/O and cheaper storage, and compressed data is often **faster** to read, because there are fewer bytes to read.

WORKED EXAMPLE: BROTLI AT THE CDN

A public JSON API sits behind a CDN. Enable Brotli at the CDN edge: browsers that send `Accept-Encoding: br` get Brotli, older clients get gzip. The origin spends no CPU on it, response sizes drop by around **80%**, and mobile load times improve noticeably.

5 When it's not worth it



Figure 3. Four cases where compression costs more than it saves.

Already-compressed data (JPEG, PNG, video, zip) won't get smaller. Tiny responses, under about 1 KB, can even grow. CPU-bound servers may get slower. And over HTTPS, compressing secrets together with attacker-controlled input can leak data through response sizes, as in the **BREACH** attack: don't compress responses that reflect user input alongside tokens, or mask the tokens. So measure, rather than turning it on everywhere blindly.

COMMON MISTAKES

- Gzip on images and video at the proxy.
- Brotli level 11 on dynamic responses: huge CPU cost per request.
- Forgetting `Vary: Accept-Encoding`, so a CDN serves compressed bytes to a client that can't read them.

6 Key terms

Compression ratio: original size ÷ compressed size

Brotli: better ratio for text, all modern browsers

Accept / Content-Encoding: client offers, server declares the encoding

BREACH: attack that infers secrets from compressed sizes

gzip: universal, decent ratio

zstd: very fast, excellent ratio, storage and internal

Parquet: columnar file format that compresses well

7 Interview questions

Q1. When is compression not worth it? BASIC

When data is already compressed, like images and video, when payloads are tiny, when the server is CPU-bound rather than network-bound, or when secrets mix with user input over HTTPS.

Q2. How do you choose an algorithm? BASIC

Brotli or gzip for HTTP text, precompressed at the highest level for static files, and zstd for storage and internal traffic.

Q3. Mobile clients complain about a slow 3 MB JSON response. What do you do? SCENARIO

Enable Brotli/gzip at the CDN or proxy, and also shrink the payload: paginate, return only needed fields, maybe switch the internal path to a compact format.

Q4. Your logs cost too much to store. Options? SCENARIO

Compress with zstd (or store as Parquet with compression), tier old logs to cheaper storage, and drop or sample noisy log lines.

Q5. Why can compressed data be faster to read from disk? TRICKY

Reading is often I/O-bound; fewer bytes to read means less I/O time, and fast decompressors like zstd or LZ4 decode faster than the disk delivers raw bytes.

Q6. How does BREACH work, roughly? TRICKY

If a response contains a secret and attacker-controlled text, the attacker guesses characters: a correct guess compresses better, so the response gets slightly smaller, which they can observe even over HTTPS.

Next: SD-J-008 · Serialization Formats backendroadmap.com